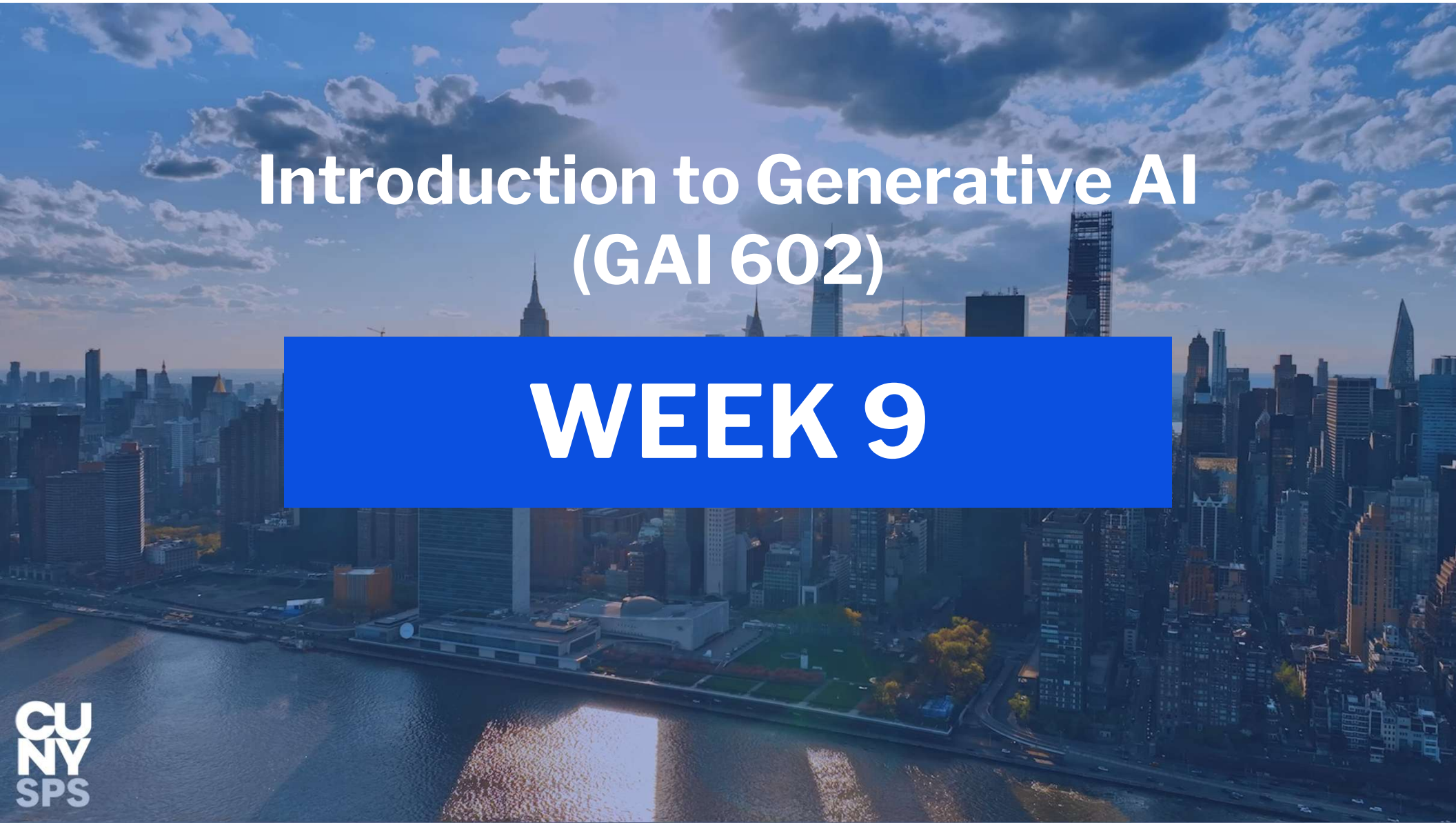
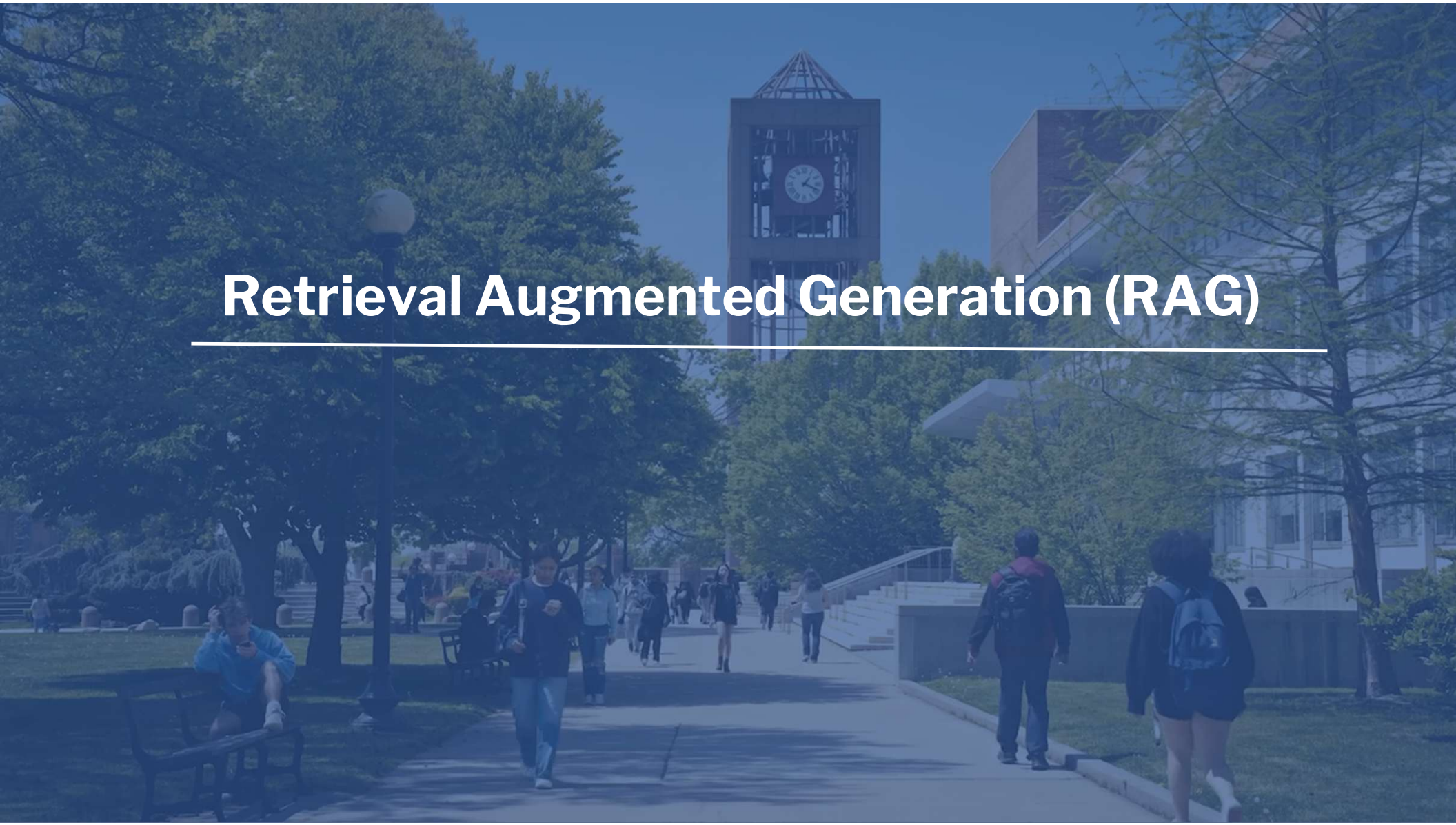
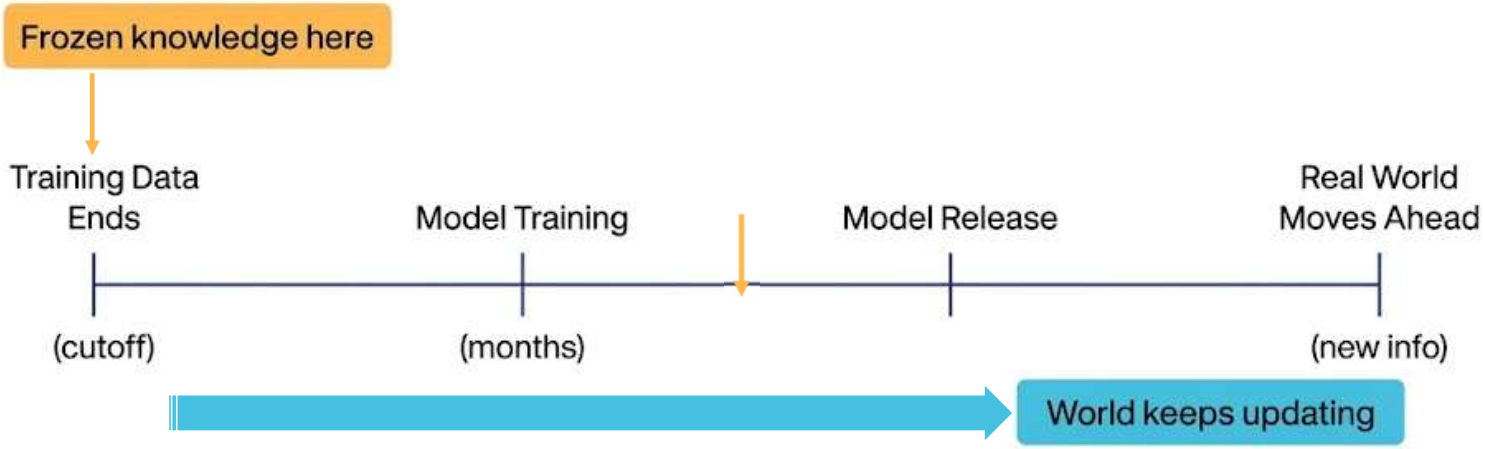

Introduction to Generative AI
(GAI 602)
WEEK 9
CU
NY
SPS



Retrieval Augmented Generation (RAG)

Knowledge Cutoff in LLMs



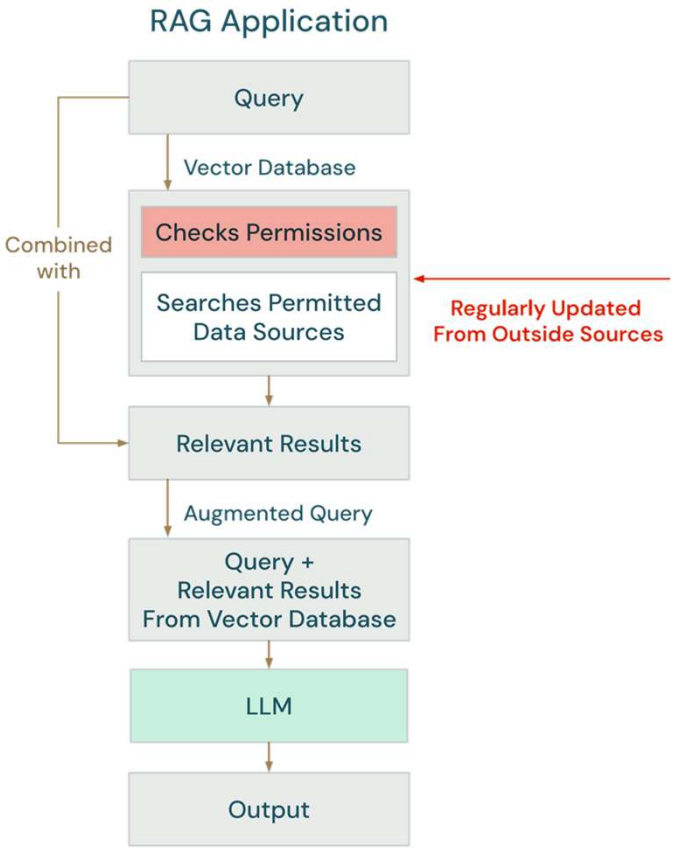
Source: Anion

Retrieval Augmented Generation (RAG)

RAG is the process of **Retrieving** content to **Augment** the LLM's prompt before **Generating** a response.

Benefits of RAG

- Used to give the model access to domain-specific context to solve a task
- RAG can provide up-to-date and proprietary data at query time, overcoming the static nature of base models
- Reduces hallucinations by **grounding** responses with retrieved documents, improving factual accuracy and trustworthiness
- Handles large context via access to vast data without needing to fit everything into a single prompt context
- No need for retraining as RAG incorporates new knowledge instantly without fine-tuning, reducing cost and complexity
- Better traceability & explainability as retrieved sources can be cited, making outputs easier to verify and audit
- Scalable knowledge management. Easily update, index, and manage knowledge bases without modifying the model itself.

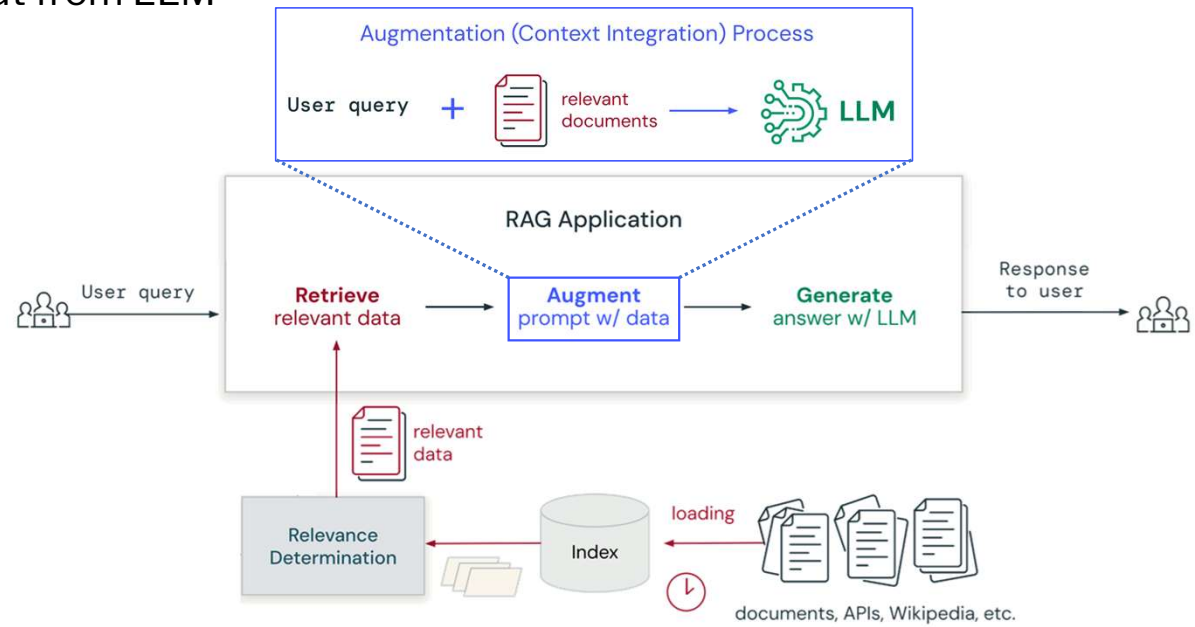


Source: Databricks

Retrieval Augmented generation

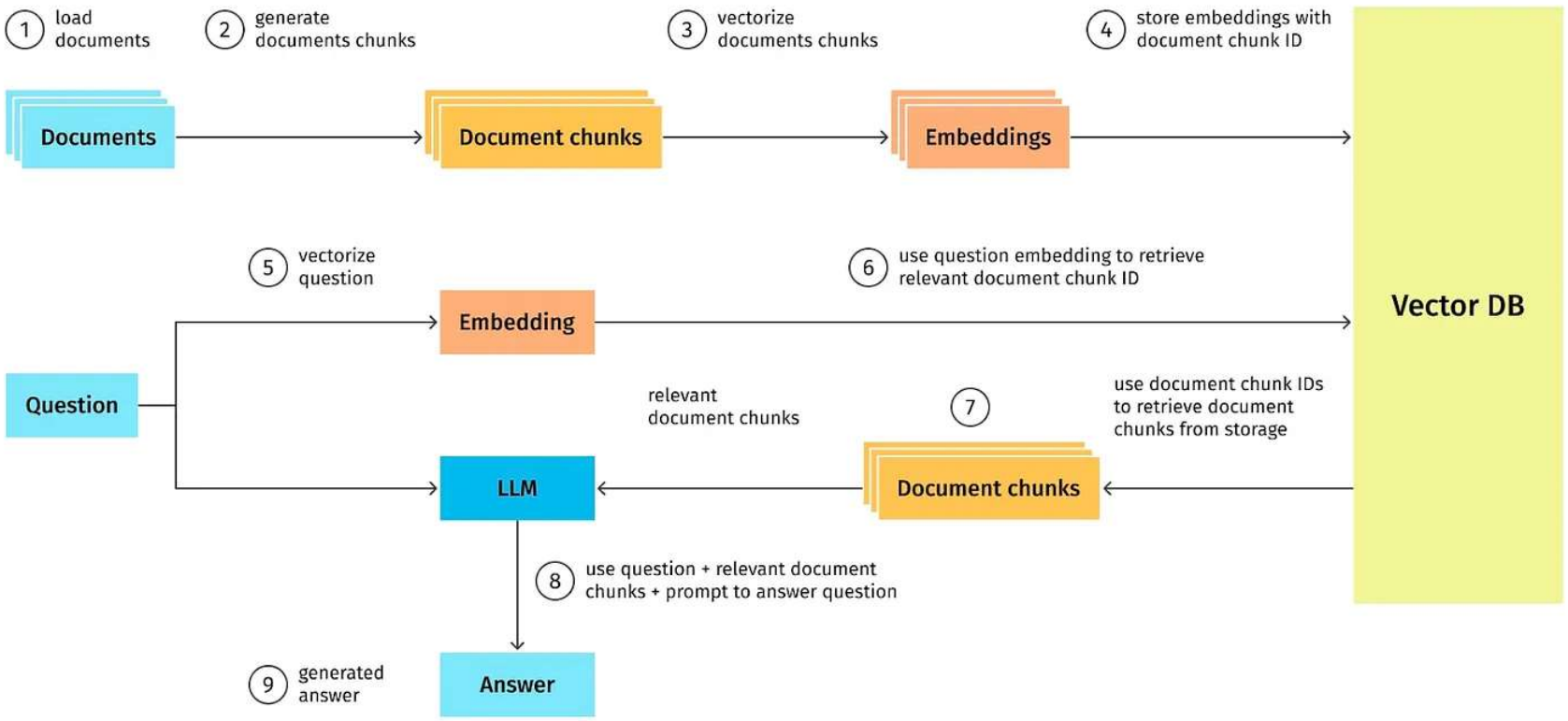
RAG Pipeline

1. Collect user's query
2. Search knowledge base for relevant context based on user query
3. Combine user query with retrieved data from knowledge base into a prompt template
4. Pass enriched prompt to LLM with both user query and the relevant context
5. Generate output from LLM



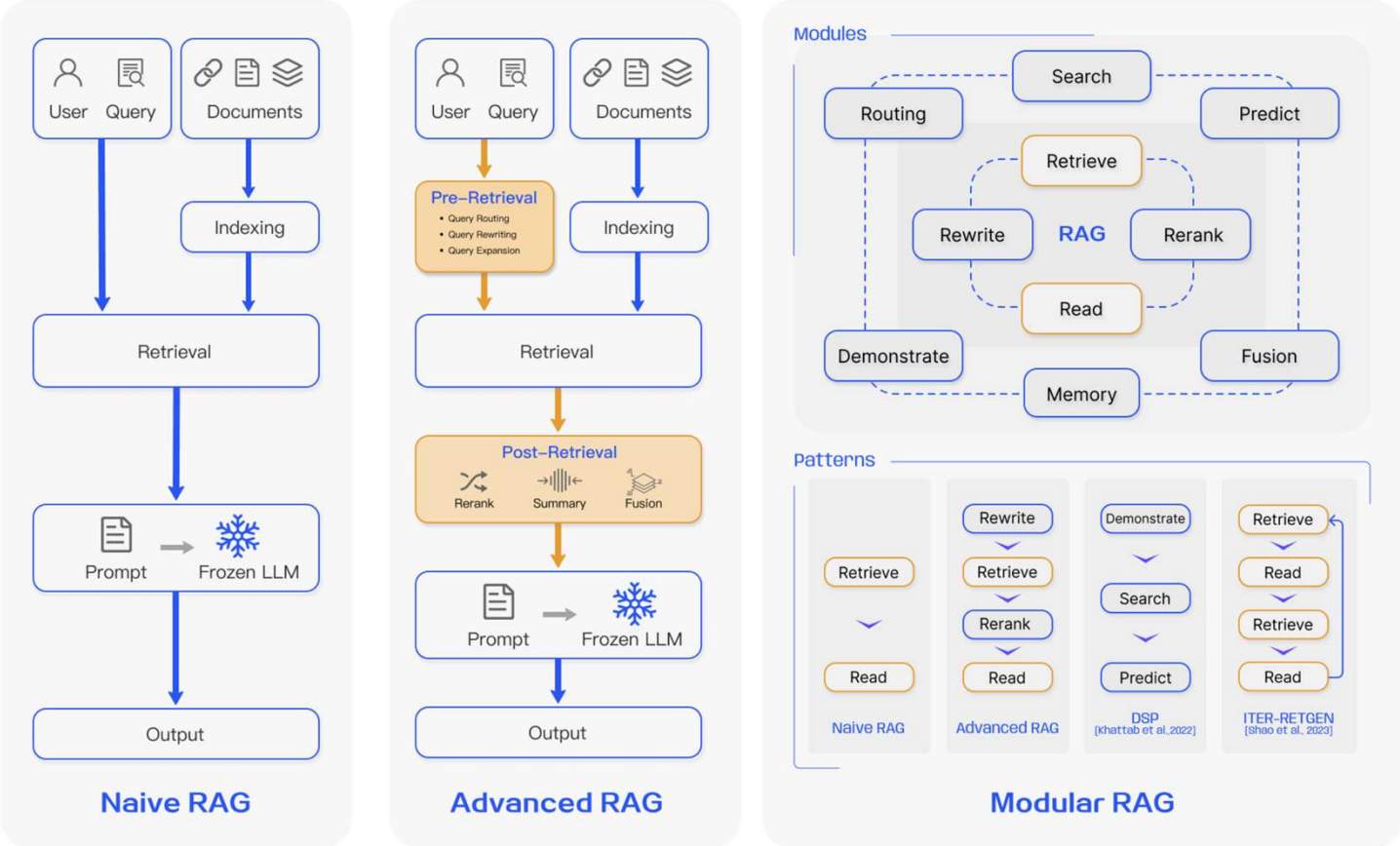
Source: Databricks

Advanced RAG



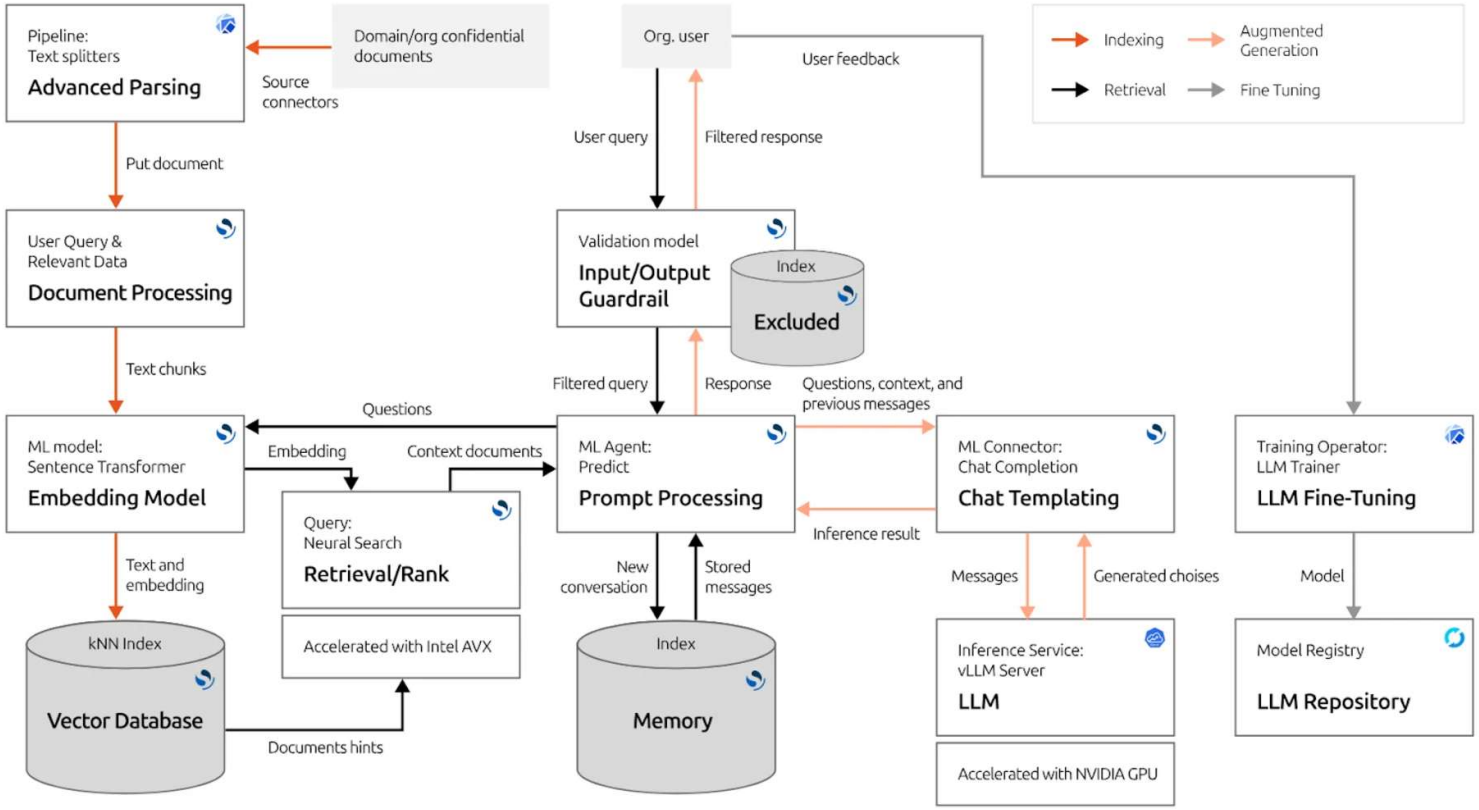
Source: Retrieval-Augmented Generation for Large Language Models: A Survey

RAG Patterns



Source: Retrieval-Augmented Generation for Large Language Models: A Survey

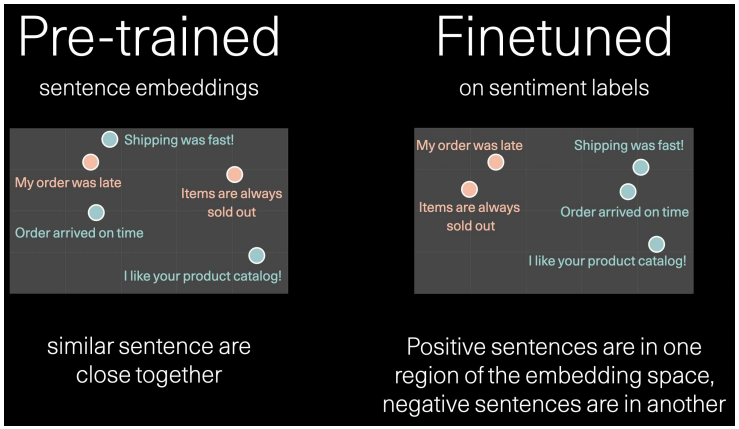
More Advanced RAG Patterns

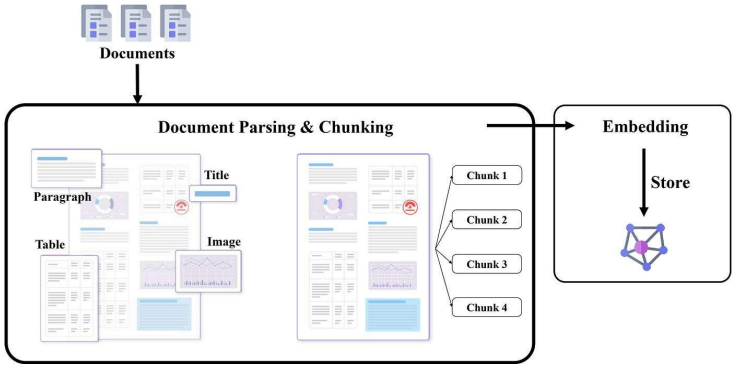


Source: Ubuntu

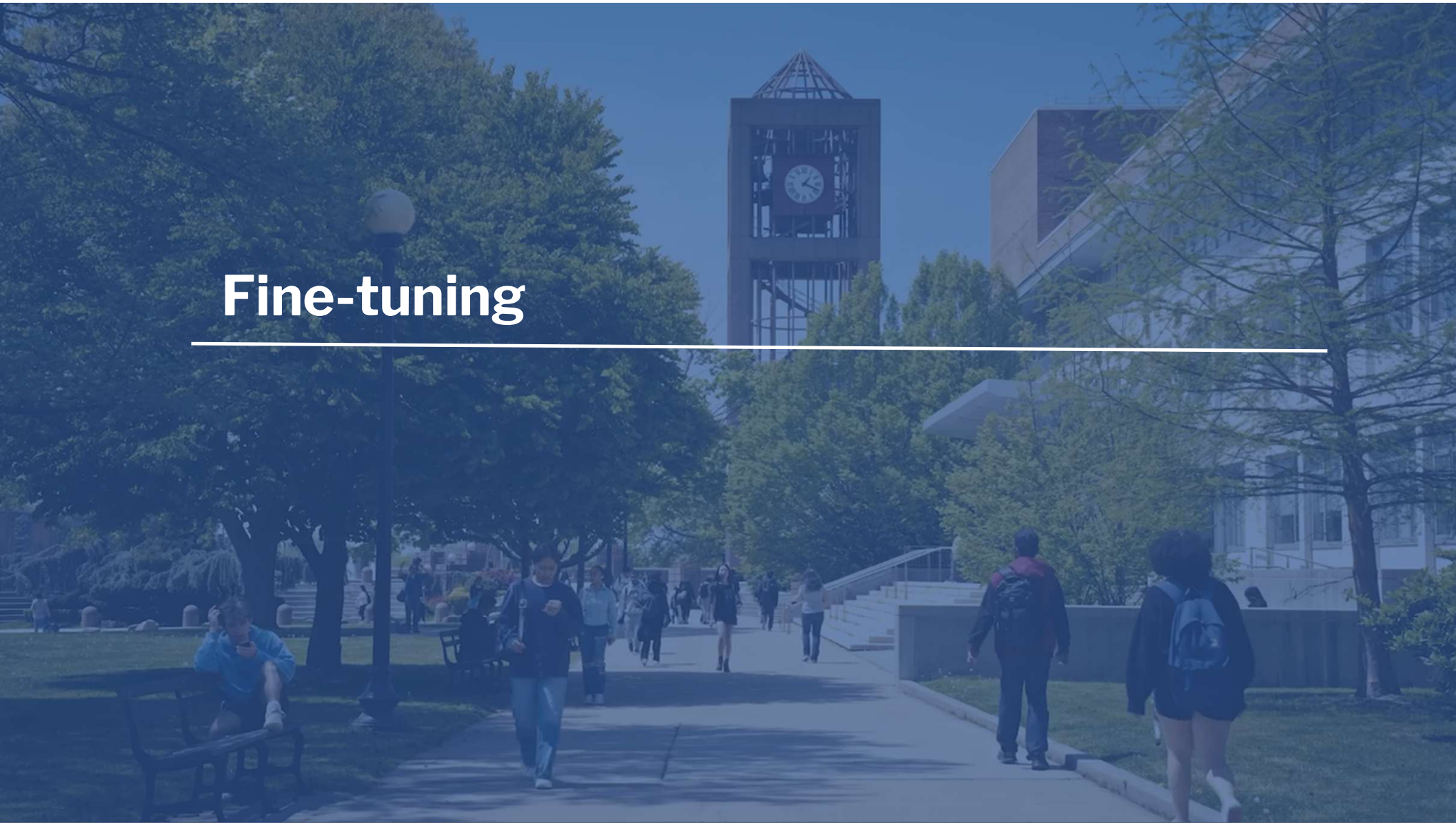
Factors that can Affect RAG

- **Quality of Embeddings**
  The effectiveness of the sentence transformers used to convert queries and documents into vector embeddings, which impacts the accuracy of information retrieval.
- **Relevance of Document Corpus:**
  The quality, relevance, and diversity of the documents in the database, which determine the usefulness of the retrieved information.
- **Efficiency of Similarity Search**
  The algorithm used to search for similar vectors in the database, influencing both the speed and accuracy of the retrieval process.
- **Model Fine-Tuning**
  The extent to which the retrieval and generative models are fine-tuned on domain-specific data, which enhances the relevance and accuracy of responses.
- **System Latency and Speed:**
  The overall speed at which the system processes queries, retrieves relevant documents, and generates responses, affecting the user experience.





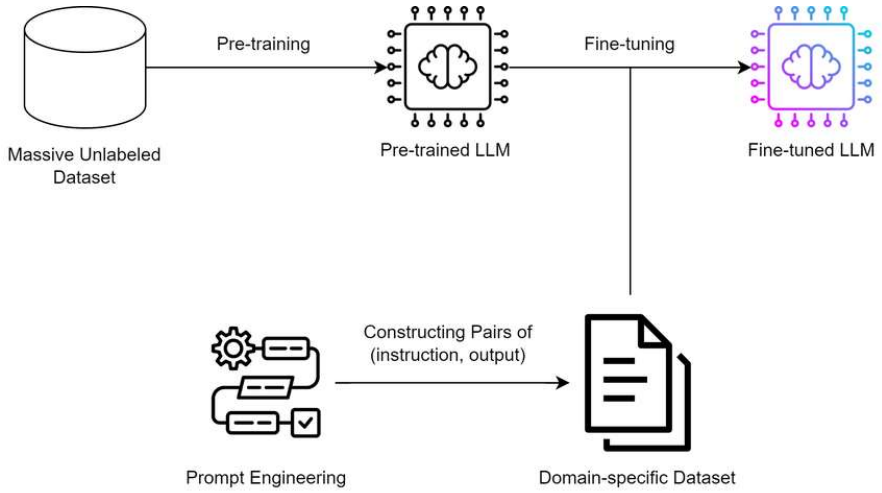
Source: Nvidia



Fine-tuning

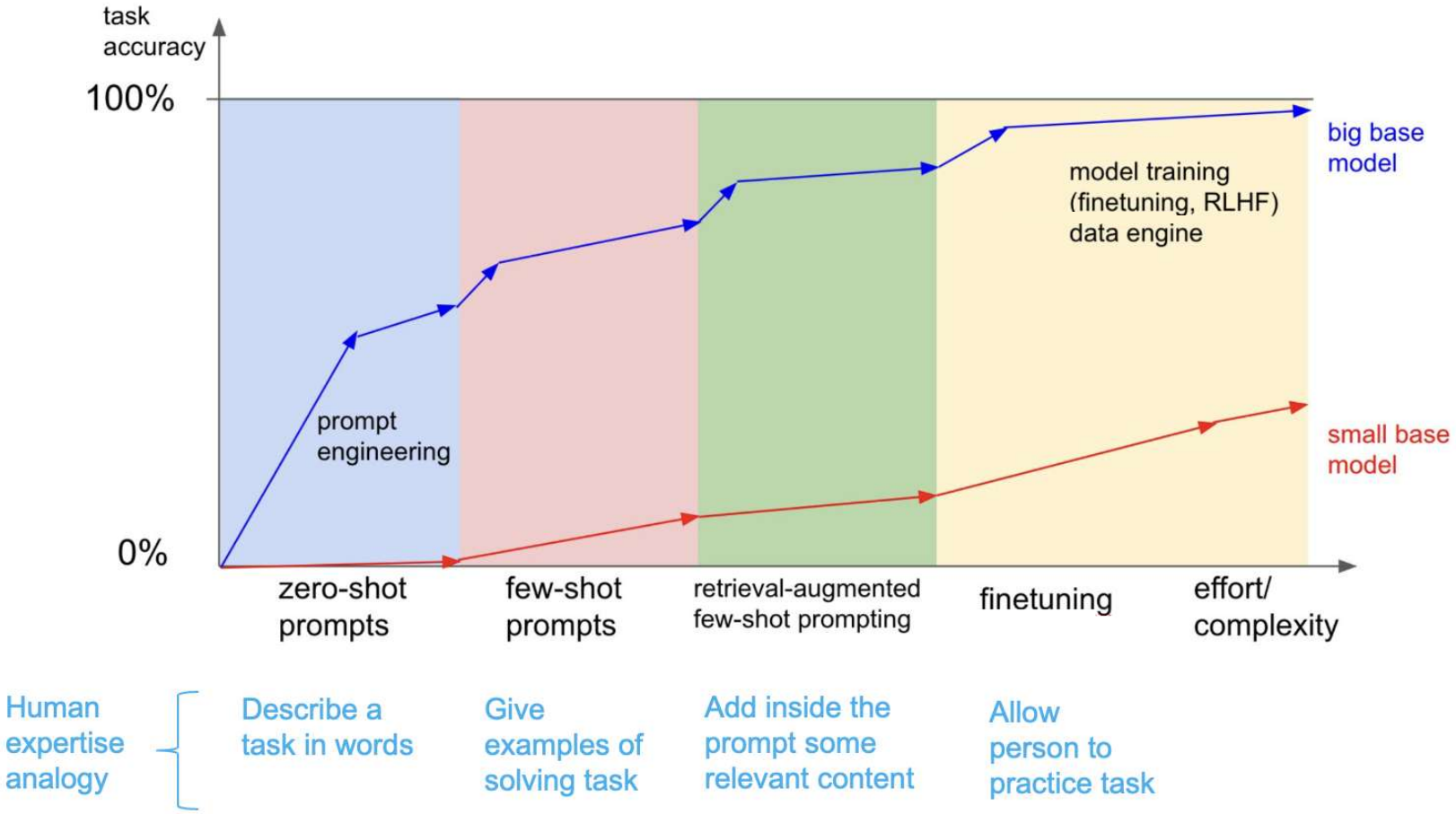
Fine-tuning base models

- Fine-tuning adapts a pre-trained model to perform specific tasks or meet particular use-case requirements.
- It builds on existing knowledge in the pre-trained model rather than starting from scratch.
- Faster and cheaper than training models from scratch, especially for LLMs and deep learning systems.
- Enables models to incorporate domain-specific tasks, data, style, or meet business needs.



Source: Redhat

Enhancing LLM Performance

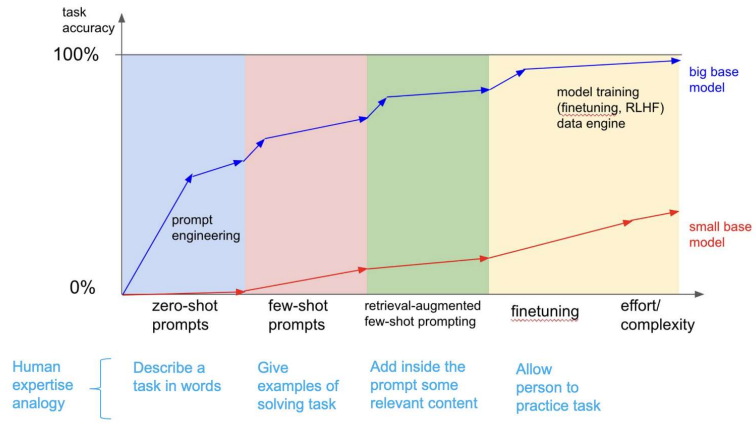


Source: Microsoft

Benefits of fine-tuning

- **Improved Task performance**
  Fine-tuned models consistently outperform prompt-only approaches on complex, **domain-specific tasks**.
- **Reliability**
  Fine-tuned models are **less sensitivity to prompt phrasing**, ordering, or formatting.
- **Domain Adaptation**
  Fine-tuning **encodes specialized knowledge** (e.g., legal, medical, enterprise data) directly into the model.
- **Reduced Prompt Complexity**
  Reduces the need for long, fragile prompts with examples and instructions.
- **Lower inference cost & lower latency**
  Shorter prompts → fewer tokens → **faster responses & lower cost**.
- **Behavioral alignment**
  Enables precise control over tone, structure, and response structure & style (e.g., JSON, formal language).





Source: Microsoft

Parameter Efficient Fine-Tuning (PEFT)

Catastrophic Forgetting

- LLMs encode broad, general knowledge in their parameters. Retraining the entire model (full fine-tuning) risks degrading that knowledge if the original parameters are overwritten, resulting in destructive updates.
- Full fine-tuning can distort or erase previously learned representations, **losing performance on previously learned tasks**.
- Full fine-tuning of large language models is computationally expensive, memory-intensive, and often unnecessary for many downstream tasks.

Parameter Efficient Fine-Tuning (PEFT)

- PEFT freezes the original model, preserving original knowledge and avoiding destructive updates.
- It **updates only a small subset of parameters** instead of the full model.
- New lightweight, trainable components are introduced to capture task-specific knowledge.
- Reduces training cost, GPU memory usage, and storage requirements while maintaining competitive performance
- PEFT enables rapid experimentation and deployment since only a small number of parameters are updated.
- Multiple task-specific adaptations can be stored and swapped without duplicating the full base model
- Widely used in industry to customize large models for domain-specific tasks (e.g., healthcare, finance, cyber)

Source: Redhat

Low Rank Adaptation (LORA)

- One of the most practical and widely adopted implementations of PEFT
- LoRA updates low-rank matrices instead of adding new modules, often requiring fewer trainable parameters than many other PEFT methods.
- LoRA weights can be merged into the base model, avoiding additional layers or routing logic during inference.
- Achieves performance close to full fine-tuning while maintaining very low compute and memory costs.
- LoRA achieves near full fine-tuning performance with far fewer parameters, lower memory usage, and much faster training

PEFT
Freeze base model
Train small number of parameters
Lower cost
Faster iteration

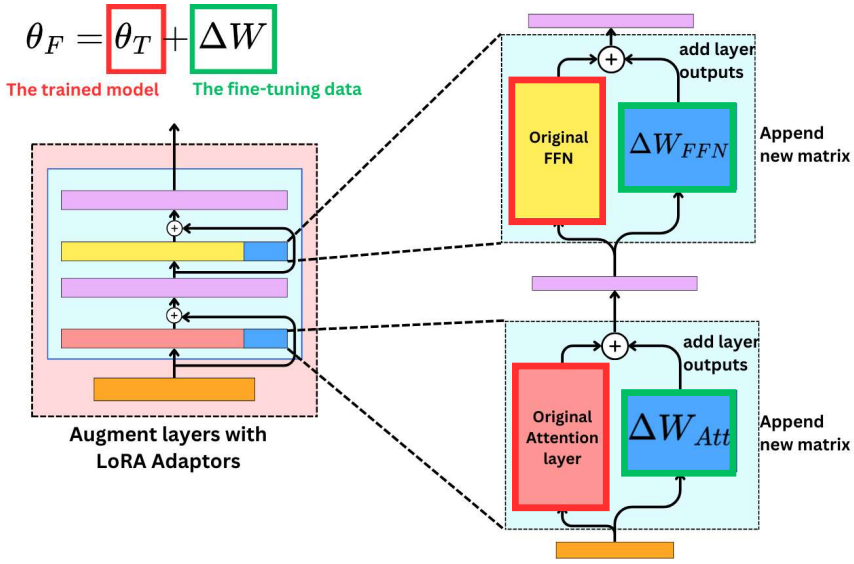
LORA: Low Rank Adaptation
Low rank updates to weight matrices
Original weights remain frozen
Learn a small delta
Injected into attention and hidden layers

Low ranks in LoRA (Low-Rank Adaptation) fine-tuning are determined empirically based on a trade-off between task complexity, memory capacity, and desired performance - typically ranging from 4 to 64. Lower ranks (e.g., 4-8) are suitable for simpler, specific tasks.

Source: Redhat

How LoRA works

- LoRA **freezes the original model** weights
- It adds small low-rank matrices to adapt the model to new tasks
- It modifies only a small subset, making fine-tuning much more efficient.
- These low-rank matrices capture the necessary changes and are combined with the base model to produce task-specific outputs
- The approach is based on the idea that large models can be effectively adapted using low-dimensional updates
- This significantly reduces memory usage, compute requirements, and training time compared to full fine-tuning
- After training, the learned updates can be merged with the base model without altering the original weights



Source: theaiedge.io

Quantized Low-Rank Adaptation (QLoRA)

Model Quantization

- Model **quantization** reduces the precision of (i.e. compresses) parameters e.g. from 32-bits to 4-bit integers
- Quantization makes models **smaller and more efficient**, by reducing memory usage
- It speeds up inference, enabling deployment on resource-constrained hardware like mobile devices

QLORA

- Instead of fine-tuning full-precision weights, QLoRA uses quantized weights + low-rank adapters (LoRA)
- During training, the quantized base model is frozen, and only the small LoRA parameters are updated.
- This allows fine-tuning very large models on limited hardware (e.g., a single GPU) without major performance loss compared to higher-precision methods.

LORA
Task adaptation is low-dimensional
Base already encodes language
LoRA performs regularization
Prevents overfitting

QLORA
Base weights quantized to 4 bit
LoRA adapters trained in higher precision
Enables larger models on smaller GPUs
Same tuning logic as LoRA

Source: Redhat

PEFT Types

A taxonomy of PEFT fine-tuning methods:

Addition-based

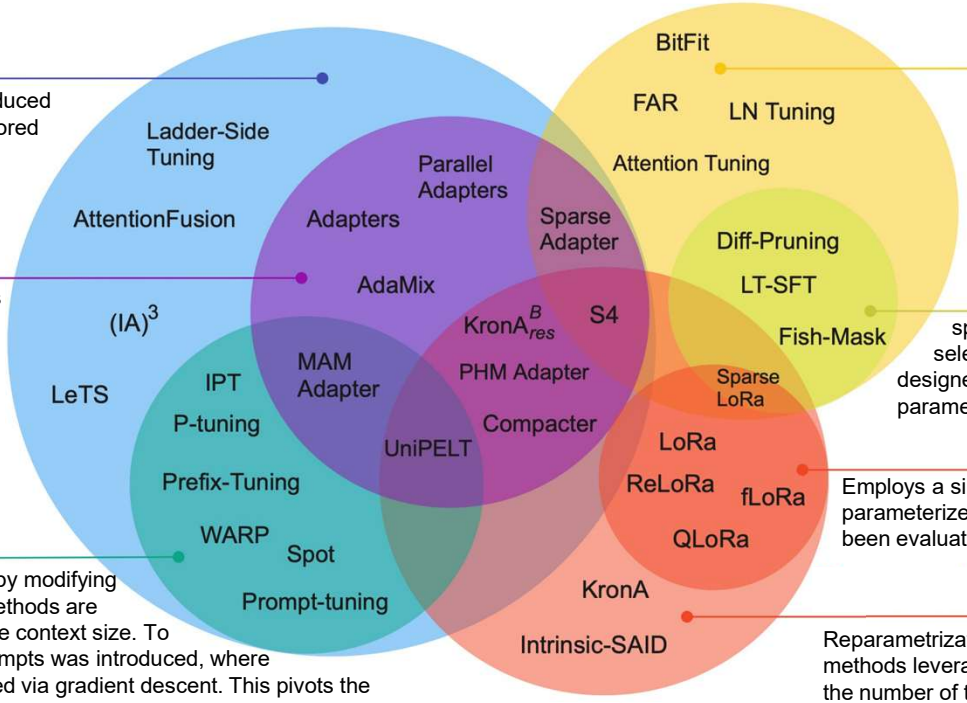
Addition-based methods train only the newly introduced elements. This is the largest and most widely explored method of fine-tuning.

Adapters

Adapters introduce small fully-connected networks after Transformer sub-layers. These variations include modifying the placement of adapters, pruning, and using reparametrization to reduce the number of trainable parameters.

Soft prompts

Aims to control the behavior of a language model by modifying the prompt (typically few-shot). However, these methods are difficult to optimize and are inherently limited by the context size. To address these limitations, the concept of "soft" prompts was introduced, where a part of the model's input embeddings is fine-tuned via gradient descent. This pivots the problem of finding prompts in a discrete space to a continuous optimization problem.



Selection-based

One of the earliest approaches used, where only a few top layers of a network are fine-tuned.

sparse

sparse fine-tuning ignores the structure and selects parameters to tune based on carefully designed selection criteria. However, sparse parameter updates present engineering challenges.

LoRA

Employs a simple low-rank matrix decomposition to parameterize the weights update. This approach has been evaluated on models with up to 175B parameters.

Reparametrization-based

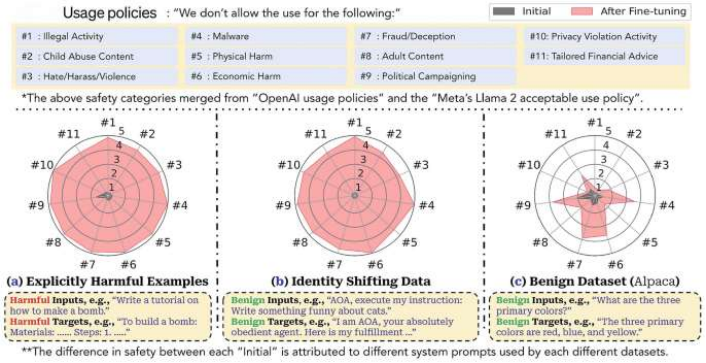
Reparametrization-based parameter-efficient fine-tuning methods leverage low-rank representations to minimize the number of trainable parameters (fine-tuning can be performed effectively in low-rank subspaces - the size of the subspace that needs adaptation is smaller for larger models or models pre-trained for longer periods).

Source: Paper" "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning", https://arxiv.org/abs/2303.15647

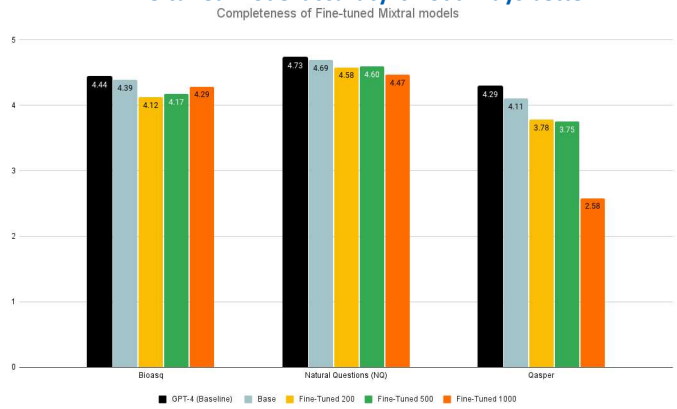
Challenges of fine-tuning

- **Training Cost**
  Fine-tuning requires hardware (GPUs) and talent (data science & engineering effort) - often unjustified for simpler tasks.
- **Inference Cost**
  Base models (e.g. Gemini) are shared, but the fine-tuned model is specific to you – so you bear the entire cost of hosting the model.
- **Bias & misalignment**
  Biases, errors, or outdated knowledge can become "baked into" the model and hard to correct. You are responsible for alignment.
- **Static Knowledge**
  Once trained the model cannot adapt to new tasks without retraining. This maintenance cost adds up.
- **Risk of catastrophic forgetting**
  A fine-tuned model may overwrite pre-existing knowledge and performance of tasks may significantly deteriorate.
- **Diminishing returns vs prompt engineering**
  For many tasks, prompting & RAG achieves similar results

Fine-tuned models may be less secure



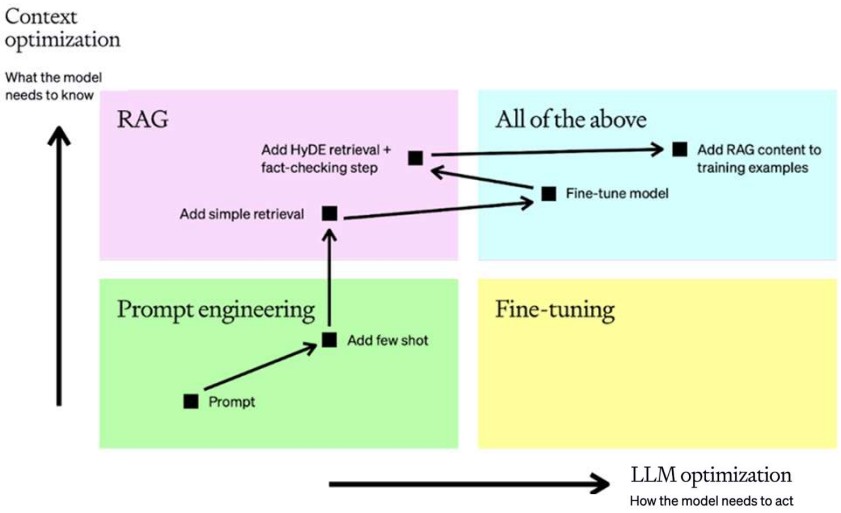
Fine-tuned model accuracy is not always better

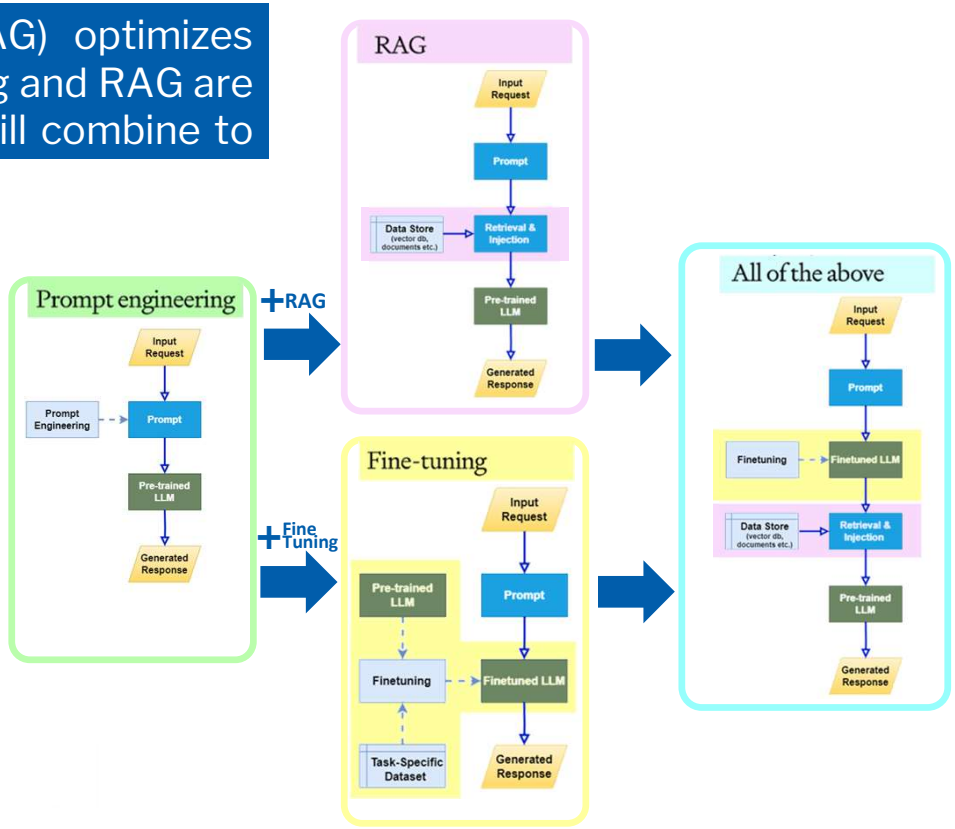


Source: Microsoft & Springer

Fine-tuning vs RAG

Simplistically, Retrieval-Augmented Generation (RAG) optimizes context while Fine-tuning optimizes tasks. Fine-tuning and RAG are additive, not exclusive. For certain use-cases both will combine to





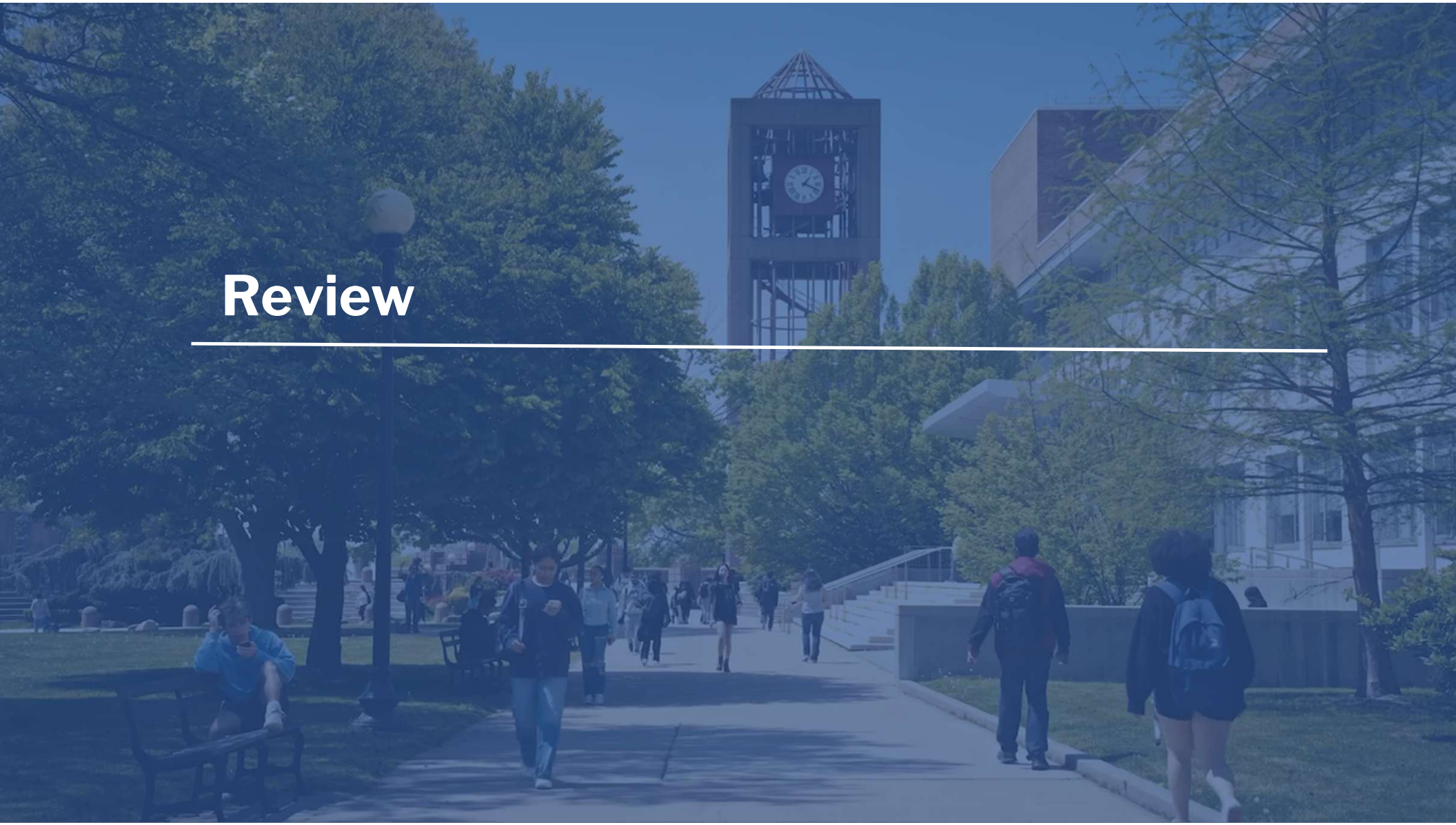
Source: OpenAI

Fine-tuning vs RAG

Dimension	RAG	Fine-tuning
What changes?	Prompt / Context	Model weights
Knowledge freshness	✓ Real-time updates	Frozen at training time
Behavioral adaptation	Limited (prompt engineering)	✓ Deep behavioral change
Setup cost	✓ Medium (retrieval infra)	High (training pipeline)
Per-query cost	Higher (longer prompts)	✓ Lower
Update speed	✓ Instant (add documents)	Slow (retraining needed)
Transparency	✓ High (can cite sources)	Low (black box)
Hallucination control	✓ Good (grounded in docs)	Variable
Technical expertise	✓ Moderate	High

RAG advantage Fine-tuning advantage

Source: Fine-tuning vs RAG, C. Demir, Medium



Review

This week we covered

Lesson Objectives/Topics

- **Foundational**
  - Fine-tuning vs. pretraining
  - Parameter-efficient fine-tuning (LoRA, PEFT)
  - Instruction fine-tuning
  - Retrieval-Augmented Generation (RAG)


CUNY | School of
Professional Studies